INSTITUTO FEDERAL
Brasília

Aula 3 – Aprofundando na programação do Arduino – Pt. 1

Marcos Santos
marcosvsantos@droidunb.com

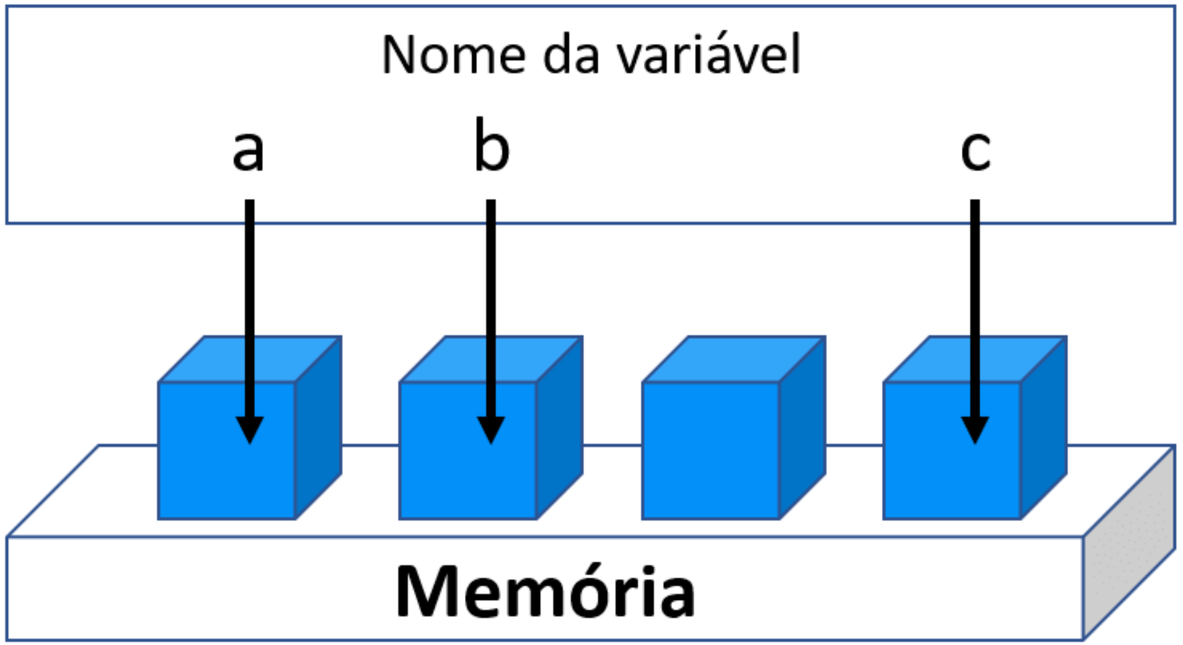
INSTITUTO FEDERAL
Brasilía

Variáveis

Variáveis

Variáveis são fundamentais para **gerenciamento dos recursos** do nosso robô, incluindo o **estado de um sensor**, a **intensidade de alguma ação** e até mesmo a **comunicação** entre seus diferentes módulos.

A analogia que usamos desde o início é entendê-las como **caixas estabelecidas na memória do nosso Arduino**. Cada caixa possui seu **próprio nome** e uma especificação do **tipo de conteúdo** que se pode colocar nela.



Tipos de variáveis

Na linguagem C, temos **restrições bem definidas sobre o conteúdo** que podemos colocar na variável. Além disso, **para cada tipo teremos uma quantidade de memória** específica que o sistema irá reservar para a variável. Temos **4 tipos principais**:

- **int** → para **números inteiros**; possuem 16 bits da memória, ou seja, podemos colocar **números entre -32.768 e 32.767** na variável
- **float** → para **números de ponto flutuante** (que utilizam as casas decimais); possuem 32 bits da memória, ou seja, podemos colocar **números entre -3,4028235*10^38 e 3,4028235*10^38**
- **bool** → só recebem **true** ou **false**. Cuidado, esse tipo **só existe de forma nativa na linguagem C do arduino**
- **char** → recebe **qualquer tipo de caractere;** deve ser escrito entre **aspas simples e só aceita um caractere**; exemplo: char letra = ‘a’;

Declaração de variáveis

No Arduino, **só declaramos uma vez a variável**. Declarar a variável é o mesmo que criar ela na memória, **passando o parâmetro do tipo dela e de seu nome**. Sempre que quisermos colocar algum valor dentro da variável que criamos, é só escrever o **nome da variável = valor atribuído**; lembre-se, isso pode ser feito **na mesma linha da declaração ou em qualquer momento depois de declará-la**, **mas nunca antes**.

Arquivo Editar Sketch Ferramentas Ajuda

sketch_apr07a §

```
int a; //a é uma variável global

void setup() {
  float b = 0; //b é uma variável local do void setup()
}

void loop() {
  bool c = false; //c é uma variável local do void loop()

  c = true;
}
```

Regras para nomear variáveis: começar com letras minúsculas, não usar caracteres especiais (com exceção do underline) ou espaço e preferir o método de digitação "lowerCamelCase"

Declaração de variáveis

Se declararmos uma variável **antes das funções void setup() e void loop()**, ela **poderá ser usada tanto no setup() quanto no loop()**, sendo chamada **variável global**. Se declararmos no **setup(), só poderá ser usada no setup()**, sendo uma **variável local**. O mesmo ocorre se declararmos no **loop(), só poderá ser usada no loop()**. Ficará mais claro o porquê disso quando estudarmos funções no C.

Arquivo Editar Sketch Ferramentas Ajuda

sketch_apr07a §

```
int a; //a é uma variável global

void setup() {
  float b = 0; //b é uma variável local do void setup()
}

void loop() {
  bool c = false; //c é uma variável local do void loop()

  c = true;
}
```

Podemos escrever qualquer coisa no C sem dar erro no código se colocarmos // antes. Chamamos isso de comentário e ele é ignorado pelo compilador

INSTITUTO FEDERAL
Brasília

Operações com int e float

Relações entre int e float

Se colocarmos um número int dentro de uma variável float, ele será tratado como float, adicionando casas decimais com valor 0. Se fizermos o contrário, as casas decimais do número float serão ignoradas e ele será tratado como int. Isso vale a variável receber o valor de uma operação matemática com números dessa forma também.

Arquivo Editar Sketch Ferramentas Ajuda

sketch_apr07a §

```
int a; //a é uma variável global

void setup() {
  float b = 0; //b é uma variável local do void setup()
}

void loop() {
  bool c = false; //c é uma variável local do void loop()

  c = true;
}
```

Podemos escrever qualquer coisa no C sem dar erro no código se colocarmos // antes. Chamamos isso de comentário e ele é ignorado pelo compilador

Operações matemáticas

Temos as seguintes **operações possíveis** com a e b sendo números quaisquer:

- **a+b:** a mais b;
- **a-b:** a menos b;
- **a*b:** a vezes b;
- **a/b:** a dividido por b.

Para mais operações, precisamos importar a biblioteca “math.h”. Veremos nas próximas aulas o que isso quer dizer.

Arquivo Editar Sketch Ferramentas Ajuda

sketch_apr07a §

```
int a; //a é uma variável global

void setup() {
  float b = 0; //b é uma variável local do void setup()
}

void loop() {
  bool c = false; //c é uma variável local do void loop()

  c = true;
}
```

Podemos escrever qualquer coisa no C sem dar erro no código se colocarmos // antes. Chamamos isso de comentário e ele é ignorado pelo compilador

INSTITUTO FEDERAL
Brasília

Funções em C

O que são e como são as funções

- Funções são **blocos de códigos** que **só serão executados se forem chamados** no código principal;
- **A parte onde ficam armazenados esses códigos na função** e que realmente roda algo é chamado **“escopo”**. **As variáveis declaradas dentro de um escopo não podem ser usadas fora dele** (assim como o que ocorre quando declaramos uma variável no setup() ou loop()). Porém, **o escopo pode usar variáveis globais ou locais de onde a função foi chamada;** O escopo é como se fosse o mundinho onde as coisas da função acontecem**;**
- Elas **podem precisar receber parâmetros** para processá-los dentro do bloco de códigos ou não;
- Elas **podem retornar um valor** depois que realizar todas as suas ações ou não;
- **Se a função retornar um valor e quisermos guardá-lo** em uma variável qualquer “a”, **podemos escrever a = função()**, assim, a função será chamada, rodará suas ações e retornará o valor para “a”.

Escrita de uma Função

A função é escrita da seguinte forma:

tipo nome(parâmetros){
    ações;
}

O tipo é em relação ao **dado que a função retorna.** Se colocarmos **void**, ela não pode retornar nada. Se colocarmos, int, char, float etc, o escopo da função deve ter o elemento: **return dado;** Lembrando, **esse dado deve respeitar o tipo**.

```
void printar(float entrada){
  Serial.print("O resultado obtido eh: ");
  Serial.println(entrada);
  delay(1000);
}

int somar(int num1, int num2){
  return num1+num2;
}

void setup() {
  Serial.begin(9600);
}

void loop() {
  float resultado;
  resultado = somar(16,18);
  printar(resultado);
}
```

Escrita de uma Função

A função é escrita da seguinte forma:

tipo nome(parâmetros){
    ações;
}

A escrita do nome segue a regra que utilizamos para as variáveis. **Não utilizem nomes que já existem em funções do sistema** ou os de outras funções.

```
void printar(float entrada){
  Serial.print("O resultado obtido eh: ");
  Serial.println(entrada);
  delay(1000);
}

int somar(int num1, int num2){
  return num1+num2;
}

void setup() {
  Serial.begin(9600);
}

void loop() {
  float resultado;
  resultado = somar(16,18);
  printar(resultado);
}
```

Escrita de uma Função

A função é escrita da seguinte forma:

tipo nome(parâmetros){
    ações;
}

Ao declarar a função, se quisermos que ela receba parâmetros, **declaramos os parâmetros da mesma forma que variáveis comuns**. Se quisermos declarar mais, **colocamos uma vírgula e declaramos outro parâmetro**, e assim por diante.

```
void printar(float entrada){
  Serial.print("O resultado obtido eh: ");
  Serial.println(entrada);
  delay(1000);
}

int somar(int num1, int num2){
  return num1+num2;
}

void setup() {
  Serial.begin(9600);
}

void loop() {
  float resultado;
  resultado = somar(16,18);
  printar(resultado);
}
```

Escrita de uma Função

A função é escrita da seguinte forma:

tipo nome(parâmetros){
    ações;
}

As **ações são comandos comuns** que utilizamos no setup() ou no loop().

```
void printar(float entrada){
  Serial.print("O resultado obtido eh: ");
  Serial.println(entrada);
  delay(1000);
}

int somar(int num1, int num2){
  return num1+num2;
}

void setup() {
  Serial.begin(9600);
}

void loop() {
  float resultado;
  resultado = somar(16,18);
  printar(resultado);
}
```

INSTITUTO FEDERAL
Brasilía

Introdução ao if e else

Estrutura if

O if é uma função do sistema que que **tem como parâmetro uma expressão lógica**, algo que só pode ter como resultado **true** ou **false**. Exemplo, **a>b, a>=b, a<b, a<=b**, só podem ser **true** ou **false**. O mesmo ocorre para as expressões **a==b (a é igual a b) e a!=b (a é diferente de b)**. Elas também possuem **seu próprio escopo de ações** que **só serão realizadas se o resultado** da expressão **for true**, **caso for false, nada é feito**.

Arquivo Editar Sketch Ferramentas Ajuda

sketch_apr07b §

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int a=2;
  int b=4;

  if(a >= b){
    Serial.println(a >= b);
    delay(1000);
  }
}
```

Lembrando, false=0 e true=1

Estrutura else

Caso precisarmos que algo **rode se a expressão lógica de if for false**, podemos **usar o else**. Ele é outra função com seu próprio escopo de parâmetros. Se precisarmos que um **outro if rode se a expressão lógica do primeiro for false**, podemos usar um **else e**, em seguida, na mesma linha, **outro if** com outra expressão. Lembrando que, **para cada if, só podemos ter um else**, mas podemos ter **quantos else if quisermos, basta ter somente um if**.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int a=2;
  int b=4;

  if(a > b){
    Serial.println(a > b);
    delay(1000);
  }
  else if(a == b){}
  else{
    Serial.println(false);
    delay(1000);
  }
}
```

INSTITUTO FEDERAL
Brasilía

Façam as questões da lista de exercícios!

INSTITUTO FEDERAL
Brasilía

Parabéns pelo empenho!

Até a próxima aula!